Alternative Assessment Tool (AAT) Report on

Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Somnath Sinharoy **1BF24EC158**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

INDEX

S.NO	CONTENTS	PAGE NO
1	Arithmetic Logic Unit (ALU)	3-5
2	Shift register	6-8
3	Conclusion	9
4	Acknowledgement	10

Arithmetic Logic Unit (ALU)

GitHub Link: https://github.com/vismithajsec24-189/ADLD-AAT-2

The Arithmetic Logic Unit (ALU) is a fundamental digital circuit that performs arithmetic and logical operations on binary data. It is an important component of a processor and other digital systems, as it enables mathematical calculations and logical decision-making. The ALU accepts binary input operands, performs the operation selected by the control signals, and produces the corresponding output.

WORKING PRINCIPLE

The ALU takes two input operands, A and B, along with select signals that determine the operation to be performed. Based on the selected control code, the circuit executes the corresponding arithmetic or logical function. The resulting value is available at the output. Depending on the design, the ALU may also generate status flags such as carry, zero, overflow, and sign flags.

The ALU can be implemented using combinational logic circuits, allowing the output to respond to changes in the input operands or selection signals after the circuit's propagation delay.

CODE:

```
module ALU_8bit (
    input  [7:0] A,
    input  [7:0] B,
    input  [3:0] ALU_sel,
    output reg [7:0] ALU_Out,
    output reg Carry,
    output Zero
);

always @(*) begin
    Carry = 0;
    ALU_Out = 8'b00000000;

    case (ALU_sel)
        4'b0000: {Carry, ALU_Out} = A + B;     // Addition
        4'b0001: ALU_Out = A - B;              // Subtraction
        4'b0010: ALU_Out = A & B;              // AND
        4'b0011: ALU_Out = A | B;              // OR
        4'b0100: ALU_Out = A ^ B;              // XOR
        4'b0101: ALU_Out = ~A;                 // NOT
        4'b0110: ALU_Out = A << 1;             // Left Shift
        4'b0111: ALU_Out = A >> 1;             // Right Shift
        4'b1000: ALU_Out = ~(A & B);           // NAND
        4'b1091: ALU_Out = ~(A | B);           // NOR
        4'b1010: ALU_Out = ~(A ^ B);           // XNOR
        4'b1011: ALU_Out = (A < B) ? 8'b0000001 : 8'b0000000; // Less Than

        default: ALU_Out = 8'b00000000;
    endcase
end
assign Zero = (ALU_Out == 8'b00000000);
endmodule
```

TESTBENCH:

```
module ALU_8bit (
    input  [7:0] A,
    input  [7:0] B,
    input  [3:0] ALU_sel,
    output reg [7:0] ALU_Out,
    output reg Carry,
    output Zero
);

always @(*) begin
    Carry = 0;
    ALU_Out = 8'b00000000;

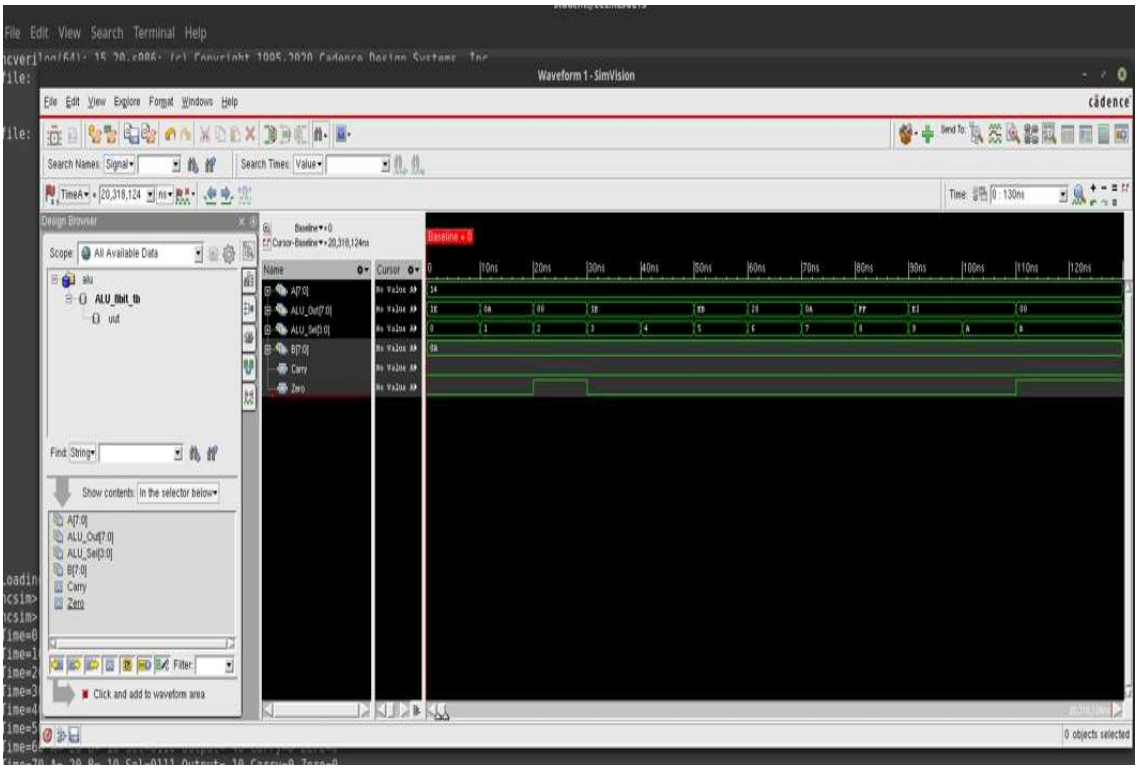
    case (ALU_sel)
        4'b0000: {Carry, ALU_Out} = A + B;     // Addition
        4'b0001: ALU_Out = A - B;              // Subtraction
        4'b0010: ALU_Out = A & B;              // AND
        4'b0011: ALU_Out = A | B;              // OR
        4'b0100: ALU_Out = A ^ B;              // XOR
        4'b0101: ALU_Out = ~A;                 // NOT
        4'b0110: ALU_Out = A << 1;             // Left Shift
        4'b0111: ALU_Out = A >> 1;             // Right Shift
        4'b1000: ALU_Out = ~(A & B);           // NAND
        4'b1091: ALU_Out = ~(A | B);           // NOR
        4'b1010: ALU_Out = ~(A ^ B);           // XNOR
        4'b1011: ALU_Out = (A < B) ? 8'b0000001 : 8'b0000000; // Less Than

        default: ALU_Out = 8'b0000000;
    endcase
end
assign Zero = (ALU_Out == 8'b00000000);
endmodule
```

```
// Declare output
initial begin
   $monitor("Time=%0d, A=%b, B=%b, Sel=%b, Y=%b, Carry=%b, Zero=%b",
            $time, A, B, ALU_Sel, ALU_Out, Carry, Zero);
end

endmodule
```

OUTPUT WAVEFORM:


File Edit View Search Terminal Help
Waveform 1 - SimVision
File Edit View Explore Format Windows Help
cadence
Search Names: Signal
Search Times: Value
Design Browser
Scope: All Available Data
alu
ALU_8bit_tb
uut
Find: String
Show contents: In the selector below
A[7:0]
ALU_Out[7:0]
ALU_Sel[3:0]
B[7:0]
Carry
Zero
Filter:
Click and add to waveform area
Name
Cursor
Baseline + 0
0
10ns
20ns
30ns
40ns
50ns
60ns
70ns
80ns
90ns
100ns
110ns
120ns
0 objects selected

Universal Shift Register

A universal shift register is a sequential digital circuit used to store binary data and perform different shifting operations. It is called a *universal* shift register because it supports multiple modes of operation, including holding the current data, shifting data to the right, shifting data to the left, and parallel loading of data. The design consists of four bits and is implemented using Verilog HDL. It uses a clock signal, an asynchronous reset, two mode selection inputs, parallel data inputs, and serial inputs for left and right shifting. The four-bit output represents the data currently stored in the register.

Working Principle

The universal shift register operates based on the positive edge of the clock signal. Whenever a rising edge of the clock occurs, the circuit checks the reset signal and the two-bit selection input to determine the operation to be performed. If the reset signal is active, all four output bits are cleared to zero immediately, without waiting for a clock edge. Otherwise, the circuit performs the operation specified by the selection inputs.

The two-bit selection input, sel, determines the mode of operation. When sel = 00, the register holds its previous value without making any changes. When sel = 01, the register performs a right-shift operation, in which the existing bits move one position to the right and the new most significant bit is supplied by serial_in_right. When sel = 10, the register performs a left-shift operation, in which the existing bits move one position to the left and the new least significant bit is supplied by serial_in_left. When sel = 11, the register performs parallel loading, in which all four bits from data_in are loaded into the register simultaneously

CODE:

```
module universal_shift_register (
    input clk,
    input rst,
    input [1:0] sel,
    input [3:0] data_in,
    input serial_in_left,
    input serial_in_right,
    output reg [3:0] q
);

always @(posedge clk or posedge rst)
begin
    if (rst)
        q <= 4'b0000;
    else
    begin
        case (sel)
            2'b00: q <= q;
            2'b01: q <= {serial_in_right, q[3:1]};
            2'b10: q <= {q[2:0], serial_in_left};
            2'b11: q <= data_in;
        endcase
    end
end

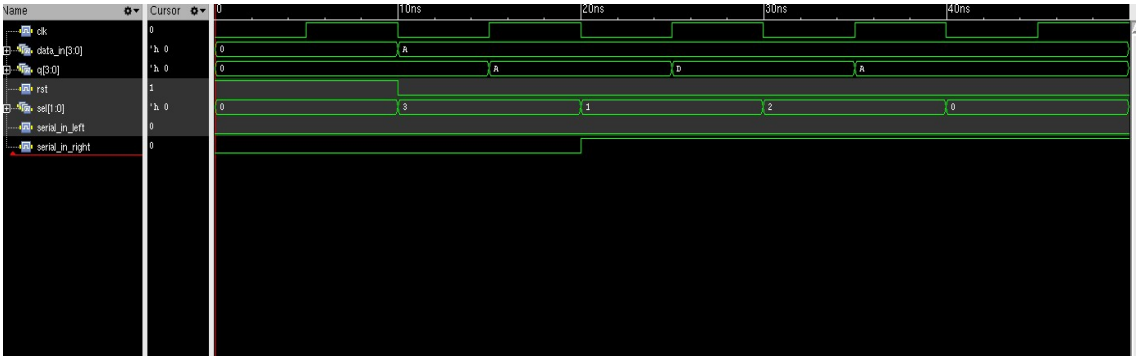
endmodule
```

TESTBENCH:

```
1
2  module tb_universal_shift_register;
3
4  reg clk, rst;
5  reg [1:0] sel;
6  reg [3:0] data_in;
7  reg serial_in_left, serial_in_right;
8  wire [3:0] q;
9
10 universal_shift_register uut (
11     .clk(clk),
12     .rst(rst),
13     .sel(sel),
14     .data_in(data_in),
15     .serial_in_left(serial_in_left),
16     .serial_in_right(serial_in_right),
17     .q(q)
18 );
19
20 // Clock generation
21 always #5 clk = ~clk;
22
23 // Waveform generation
24 initial begin
25     $dumpfile("universal_shift_register.vcd");
26     $dumpvars(0, tb_universal_shift_register);
27 end
28
29 initial begin
30     clk = 0;
31     rst = 1;
32     sel = 0;
33     data_in = 0;
34     serial_in_left = 0;
35     serial_in_right = 0;
36
37     #10 rst = 0;
38
39     // Parallel Load
40     sel = 2'b11;
41     data_in = 4'b1010;
42     #10;
43
44     // Shift Right
45     sel = 2'b01;
46     serial_in_right = 1;
47     #10;
48
```

```
36
37      #10 rst = 0;
38
39      // Parallel Load
40      sel = 2'b11;
41      data_in = 4'b1010;
42      #10;
43
44      // Shift Right
45      sel = 2'b01;
46      serial_in_right = 1;
47      #10;
48
49      // Shift Left
50      sel = 2'b10;
51      serial_in_left = 0;
52      #10;
53
54      // Hold
55      sel = 2'b00;
56      #10;
57
58      $finish;
59  end
60
61  initial begin
62      $monitor("Time=%0t sel=%b q=%b", $time, sel, q);
63  end
64
65  endmodule
66
```

OUTPUT WAVEFORM:



CONCLUSION

The Arithmetic Logic Unit (ALU) and Shift Register were designed and simulated using Cadence tools to understand the fundamental concepts of digital circuit design. The ALU demonstrates the execution of arithmetic and logical operations on binary data based on the selected control signals, while the shift register illustrates the storage and shifting of binary data between flip-flops in response to clock pulses. The simulation results help verify the functionality of both circuits and provide practical knowledge of combinational and sequential logic design. Thus, these circuits form essential building blocks of digital systems and are widely used in processors, microcontrollers, and data processing applications.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Prof. Ajaykumar Devarapalli for his valuable guidance and support throughout the completion of this Alternative Assessment Tool (AAT) report for the course Advanced Digital and Logic Design (ADLD). His guidance helped me understand the design and simulation of digital circuits using Verilog HDL and Cadence tools.

I am also thankful to the Department of Electronics and Communication Engineering, BMS College of Engineering, for providing me with the opportunity and academic environment to carry out this work on the design and verification of the Arithmetic Logic Unit (ALU) and Shift Register using Verilog HDL and Cadence simulation tools.

Finally, I would like to thank everyone who contributed directly or indirectly to the successful completion of this report.

Somnath Sinharoy
USN: 1BF24EC158